

Data Structures: Assignment-6

Name: Suraj Shah

Admission Number: I24AI012

Question 1: Write a C/C++ program to convert an infix expression to a postfix expression using a stack, handling the operators +, -, *, /, and parentheses.

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct Node {
    char data;
    struct Node *next;
}Node;

typedef struct Stack {
    Node *top;
}Stack;

void create(Stack *s) {
    s->top=NULL;
}

int isEmpty(Stack *s) {
    return s->top==NULL;
}

void push(Stack *s, char ch) {
    Node *newNode=(Node *)malloc(sizeof(Node));
    if (!newNode) {
        printf("Stack Overflow\n");
        return;
    }
    newNode->data=ch;
    newNode->next=s->top;
    s->top=newNode;
}

char pop(Stack *s) {
    if(isEmpty(s)) {
        return '\0';
    }
}
```

```

    Node *temp=s->top;
    char popped=temp->data;
    s->top=temp->next;
    free(temp);
    return popped;
}

char peek(Stack *s) {
    if (!isEmpty(s)) {
        return s->top->data;
    }
    return '\0';
}

int precedence(char op) {
    if (op == '+' || op == '-') {
        return 1;
    }
    if (op == '*' || op == '/') {
        return 2;
    }
    return 0;
}

void infixToPostfix(char *infix, char *postfix) {
    Stack st;
    create(&st);
    int i=0, j=0;

    while (infix[i]!='\0') {
        char ch=infix[i];

        if ((ch>='A' && ch <='Z') || (ch>='a' && ch<='z')) {
            postfix[j++]=ch;
        }

        else if (ch=='(') {
            push(&st, ch);
        }

        else if (ch==')') {
            while (!isEmpty(&st) && peek(&st)!='(') {
                postfix[j++]=pop(&st);
            }
            if (!isEmpty(&st)) {
                pop(&st);
            }
        }
    }
}

```

```

        else {
            while (!isEmpty(&st) && precedence(peek(&st))>=precedence(ch)) {
                postfix[j++]=pop(&st);
            }
            push(&st,ch);
        }
        i++;
    }

    while (!isEmpty(&st)) {
        postfix[j++]=pop(&st);
    }
    postfix[j]='\0';
}

int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter an infix expression: ");
    scanf("%s",infix);

    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n",postfix);

    return 0;
}

```

Output:

```

Enter an infix expression: A*(B+C*D)+E
Postfix expression: ABCD*+*E+

```